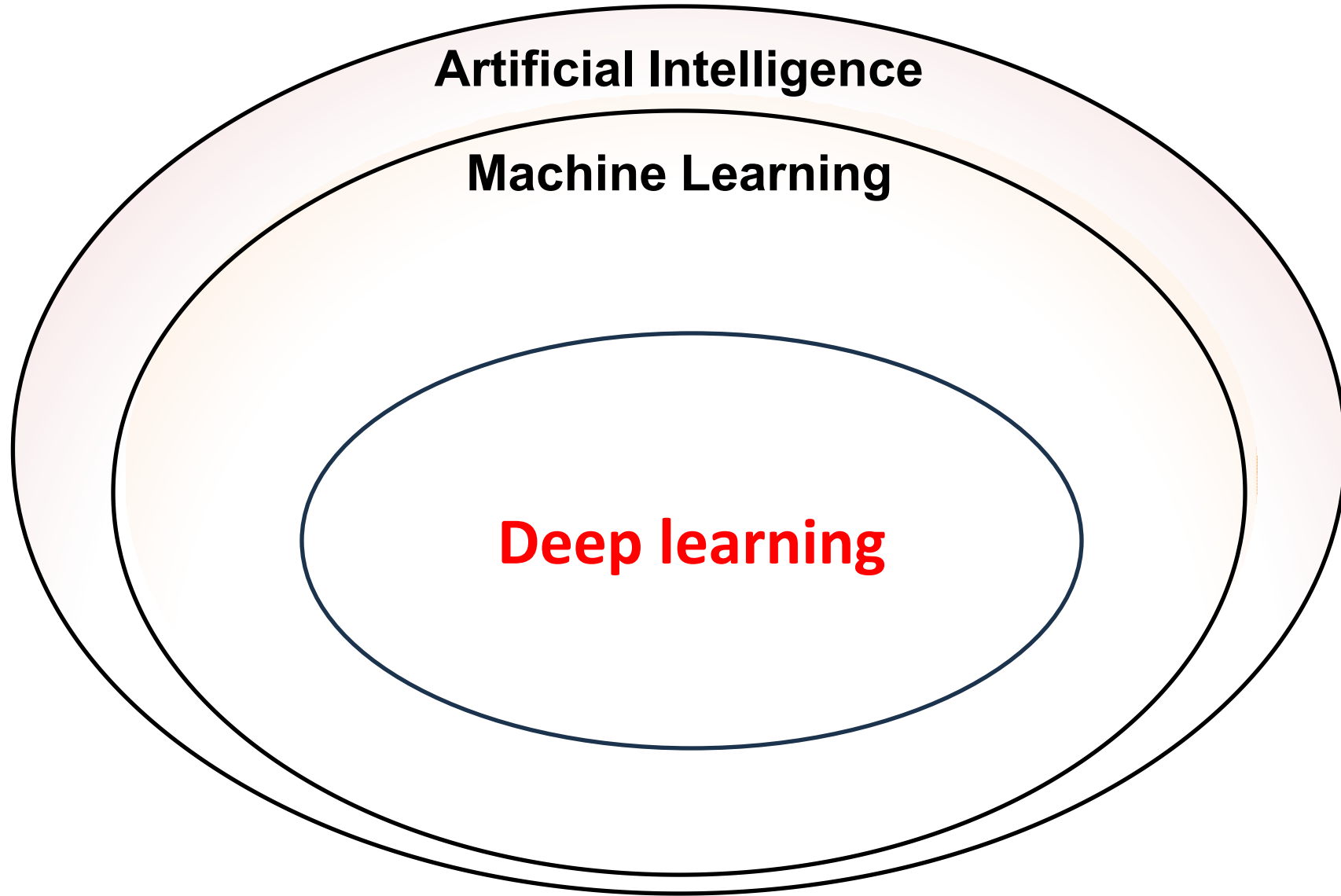
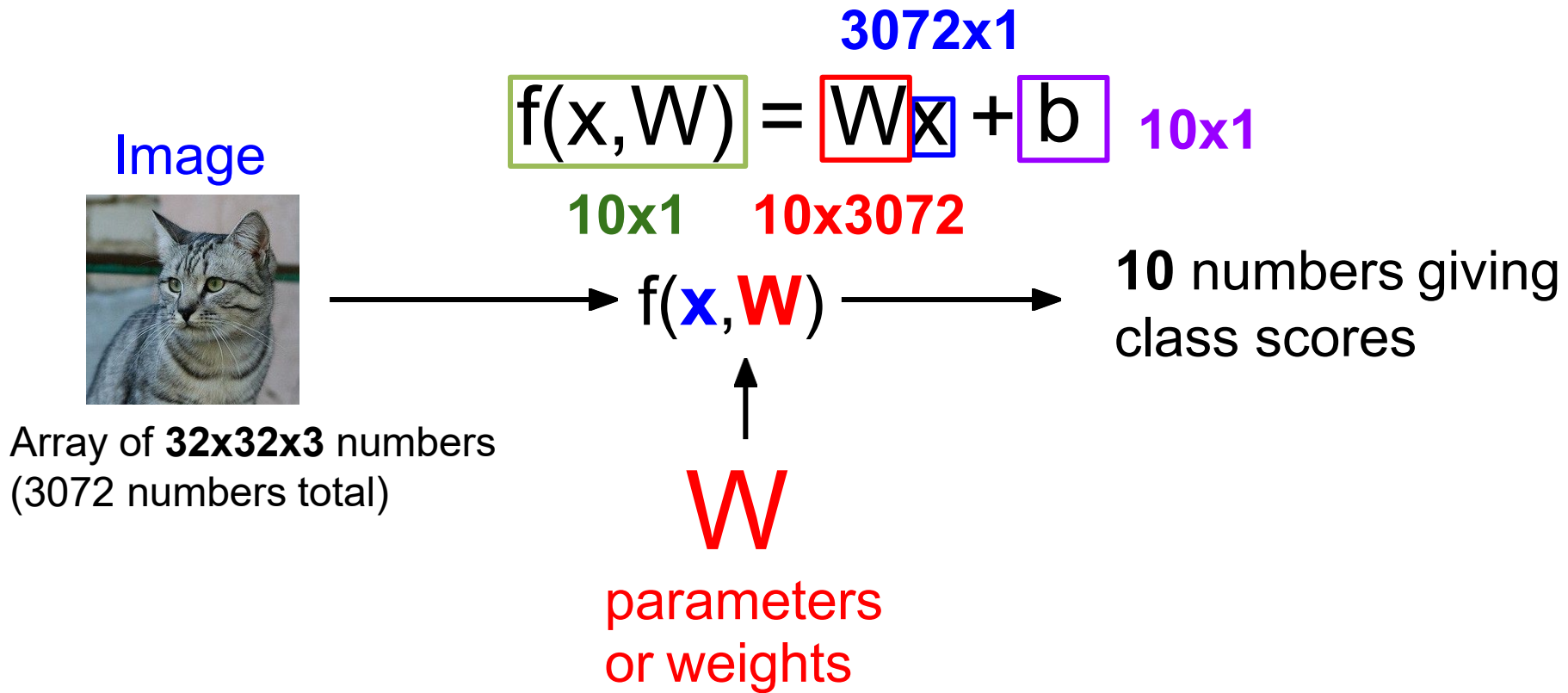


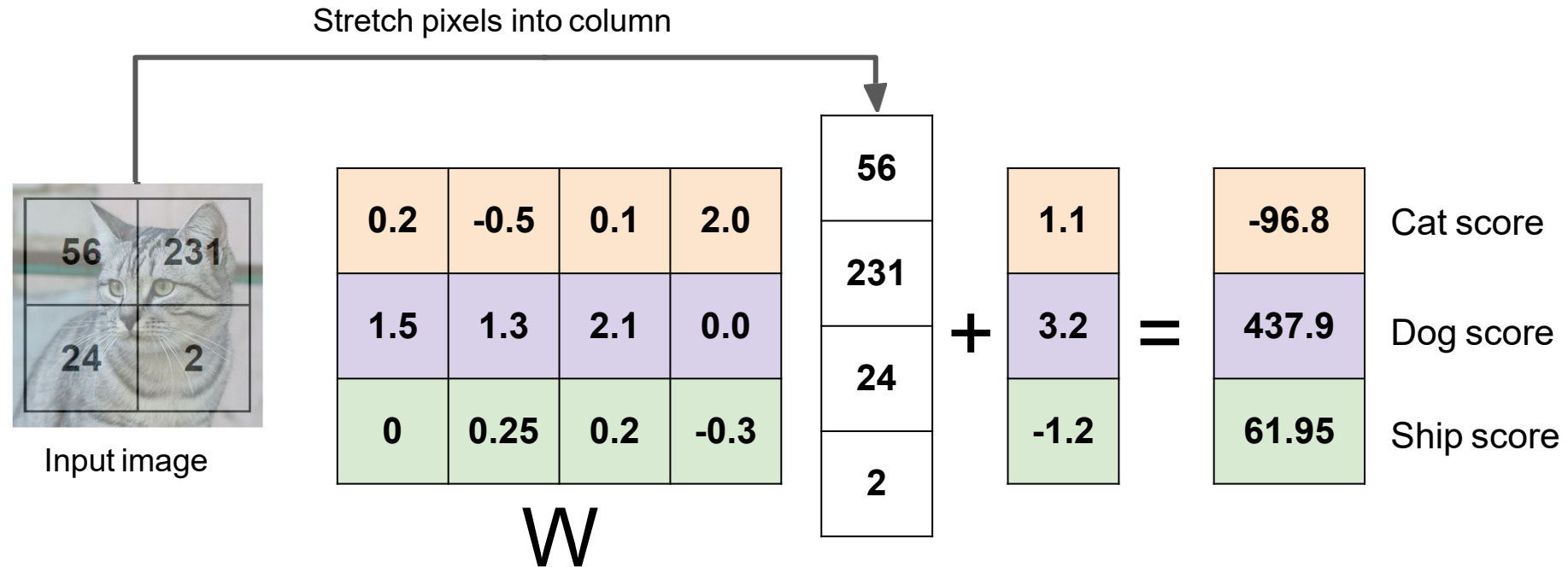
AI and Machine Learning



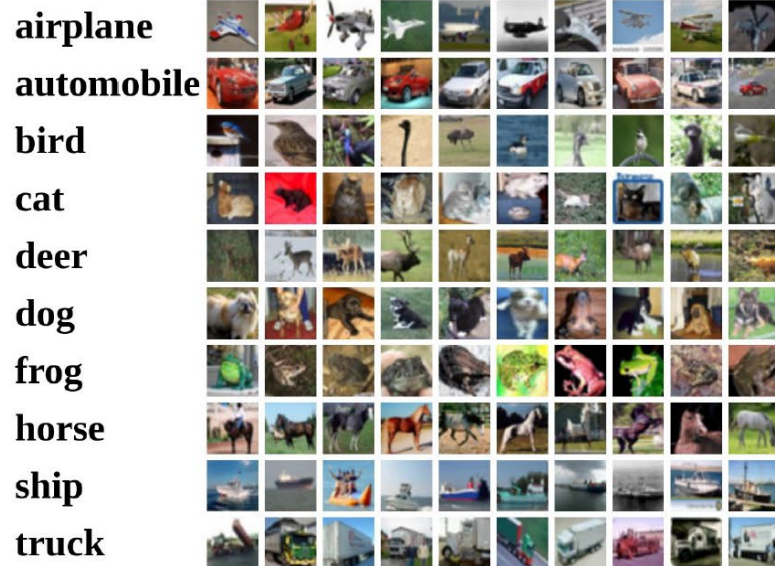
Linear Classifier



An image with 4 pixels, and 3 classes (cat/dog/ship)



Interpreting a Linear Classifier

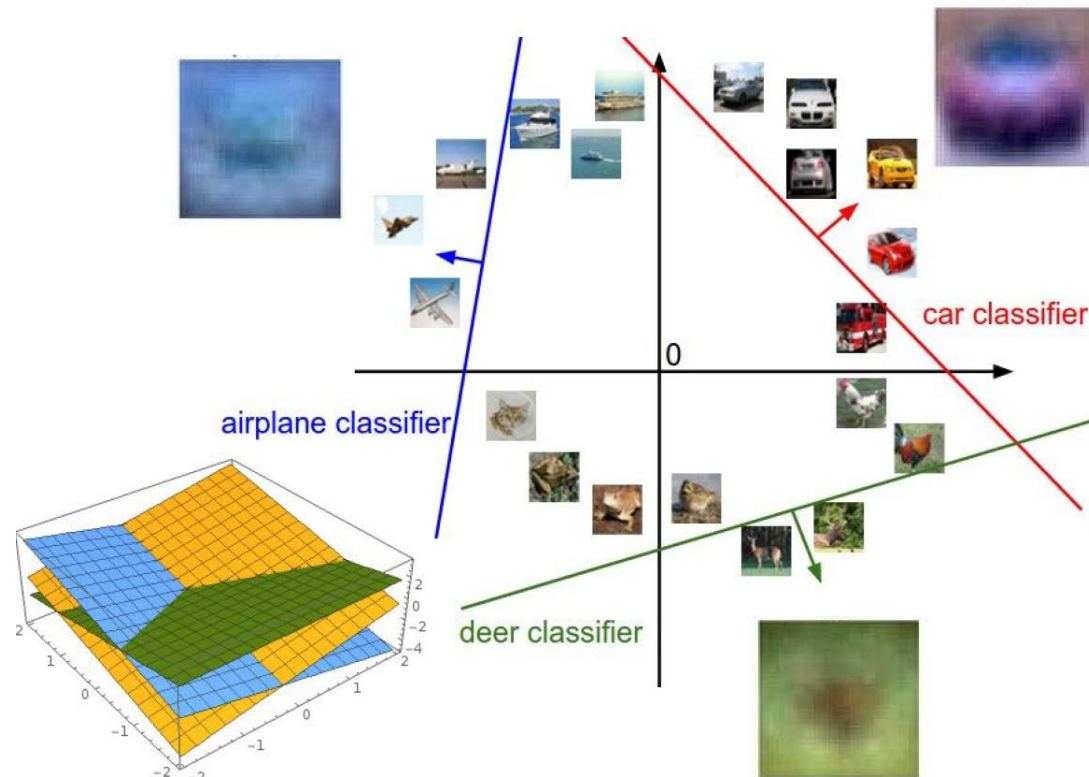


$$f(x, W) = Wx + b$$

Example trained weights
of a linear classifier
trained on CIFAR-10:



Interpreting a Linear Classifier



$$f(x, W) = Wx + b$$



Array of **32x32x3** numbers
(3072 numbers total)

Evaluating a Linear Classifier

Suppose: 3 training examples, 3 classes.

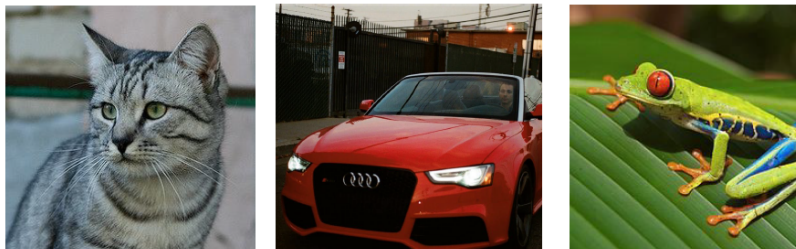
With some W the scores $f(x, W) = Wx$ are:
assuming bias 0 for simplicity



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

Evaluating a Linear Classifier

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

A **loss function** tells how good our current classifier is

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^N$$

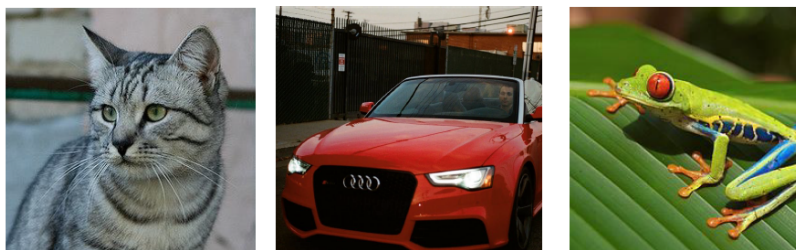
Where x_i is image and
 y_i is (integer) label

Loss over the dataset is a
sum of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Evaluating a Linear Classifier

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

Multiclass SVM loss:

hinge loss

Given an example (x_i, y_i)
where x_i is the image and
where y_i is the (integer) label,

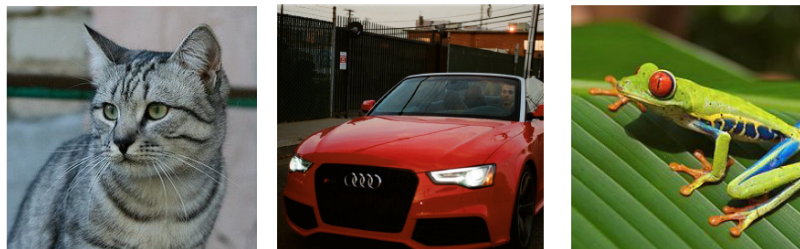
and using the shorthand for the
scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & \text{if } s_{y_i} \geq s_j + 1 \\ s_j - s_{y_i} + 1 & \text{otherwise} \end{cases}$$
$$= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Evaluating a Linear Classifier

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9		

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is the image and
where y_i is the (integer) label,

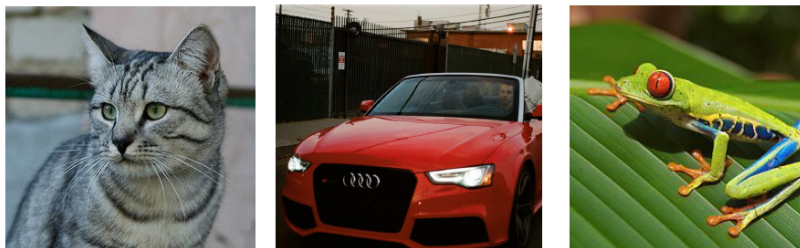
and using the shorthand for the
scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$\begin{aligned} L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ &= \max(0, 5.1 - 3.2 + 1) \\ &\quad + \max(0, -1.7 - 3.2 + 1) \\ &= \max(0, 2.9) + \max(0, -3.9) \\ &= 2.9 + 0 \\ &= 2.9 \end{aligned}$$

Evaluating a Linear Classifier

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is the image and
where y_i is the (integer) label,

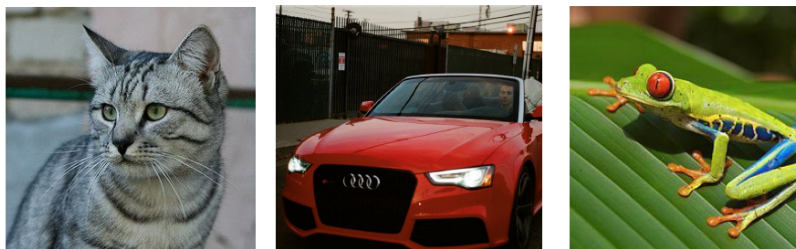
and using the shorthand for the
scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$\begin{aligned} L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ &= \max(0, 1.3 - 4.9 + 1) \\ &\quad + \max(0, 2.0 - 4.9 + 1) \\ &= \max(0, -2.6) + \max(0, -1.9) \\ &= 0 + 0 \\ &= 0 \end{aligned}$$

Evaluating a Linear Classifier

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is the image and
where y_i is the (integer) label,

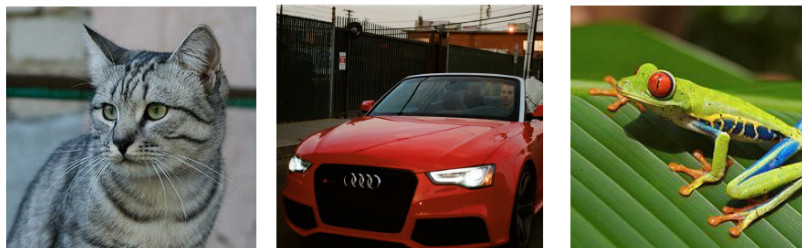
and using the shorthand for the
scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$\begin{aligned} L_i &= \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \\ &= \max(0, 2.2 - (-3.1) + 1) \\ &\quad + \max(0, 2.5 - (-3.1) + 1) \\ &= \max(0, 6.3) + \max(0, 6.6) \\ &= 6.3 + 6.6 \\ &= 12.9 \end{aligned}$$

Evaluating a Linear Classifier

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

Multiclass SVM loss:

Given an example (x_i, y_i)
where x_i is the image and
where y_i is the (integer) label,

and using the shorthand for the
scores vector: $s = f(x_i, W)$

the SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Loss over full dataset is average:

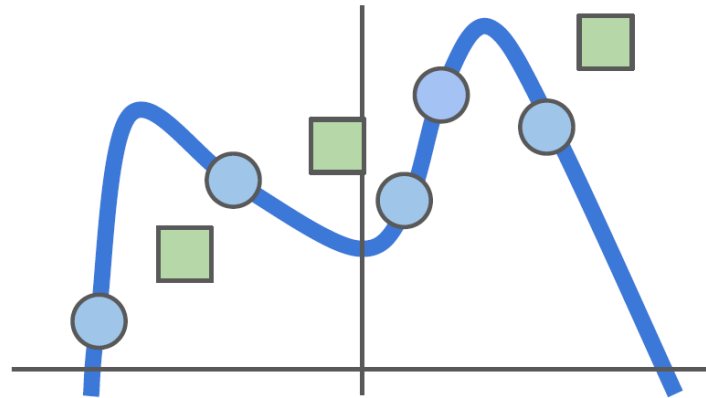
$$L = \frac{1}{N} \sum_{i=1}^N L_i$$

$$L = (2.9 + 0 + 12.9)/3 \\ = 5.27$$

Evaluating a Linear Classifier

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$

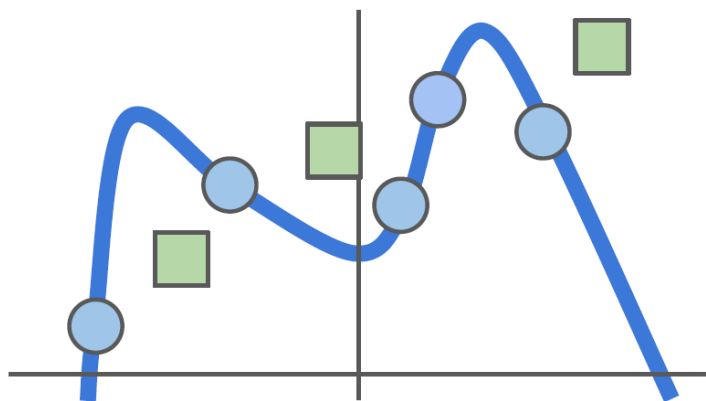
Data loss: Model predictions should match training data



Evaluating a Linear Classifier

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$

Data loss: Model predictions should match training data



Generalization of the classifier

- The Lin classifier must work also on training data **&** on general data
- Minimizing the training error (current definition of $L(W)$) is good, but it might introduce some issues
- $L(W)$ is optimized too well $\rightarrow$ the classifier works well on training data but not on general data $\rightarrow$ **OVERFITTING** [It learns noise from training data]
- Not learning general rules!
- **Regularization of the classifier** needed to avoid Overfitting and numerical issues

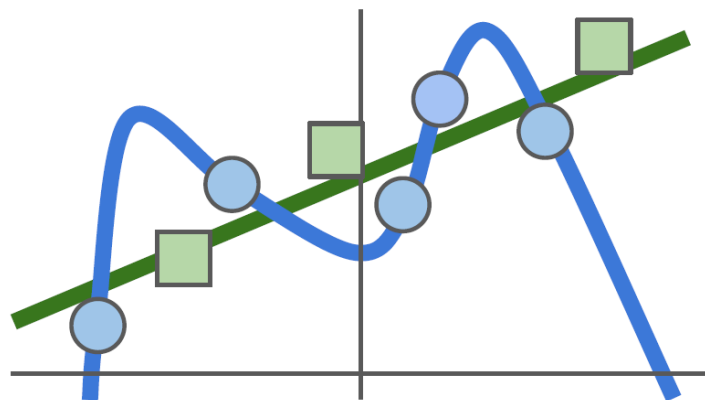
Evaluating a Linear Classifier

Regularization refers to the act of modifying a learning algorithm to favor “simpler” prediction rules to avoid overfitting

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Model should be “simple”, so it works on test data



Occam's Razor:

“Among competing hypotheses, the simplest is the best”

William of Ockham, 1285 - 1347

Training a Linear Classifier

Follow the Slope: gradient-based optimisation



The minimization problem (training) is addressed with Gradient-descent technique

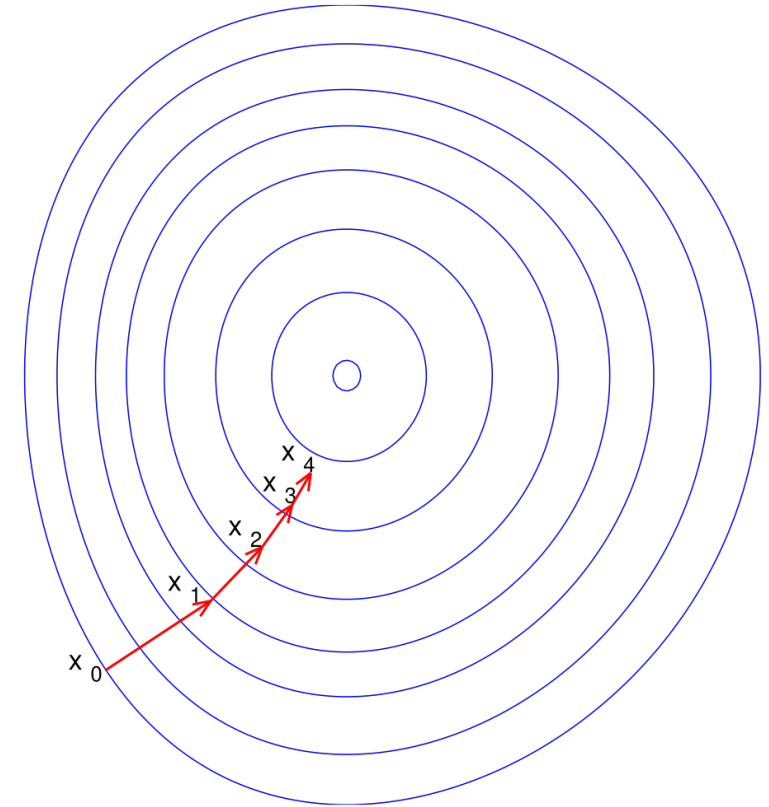
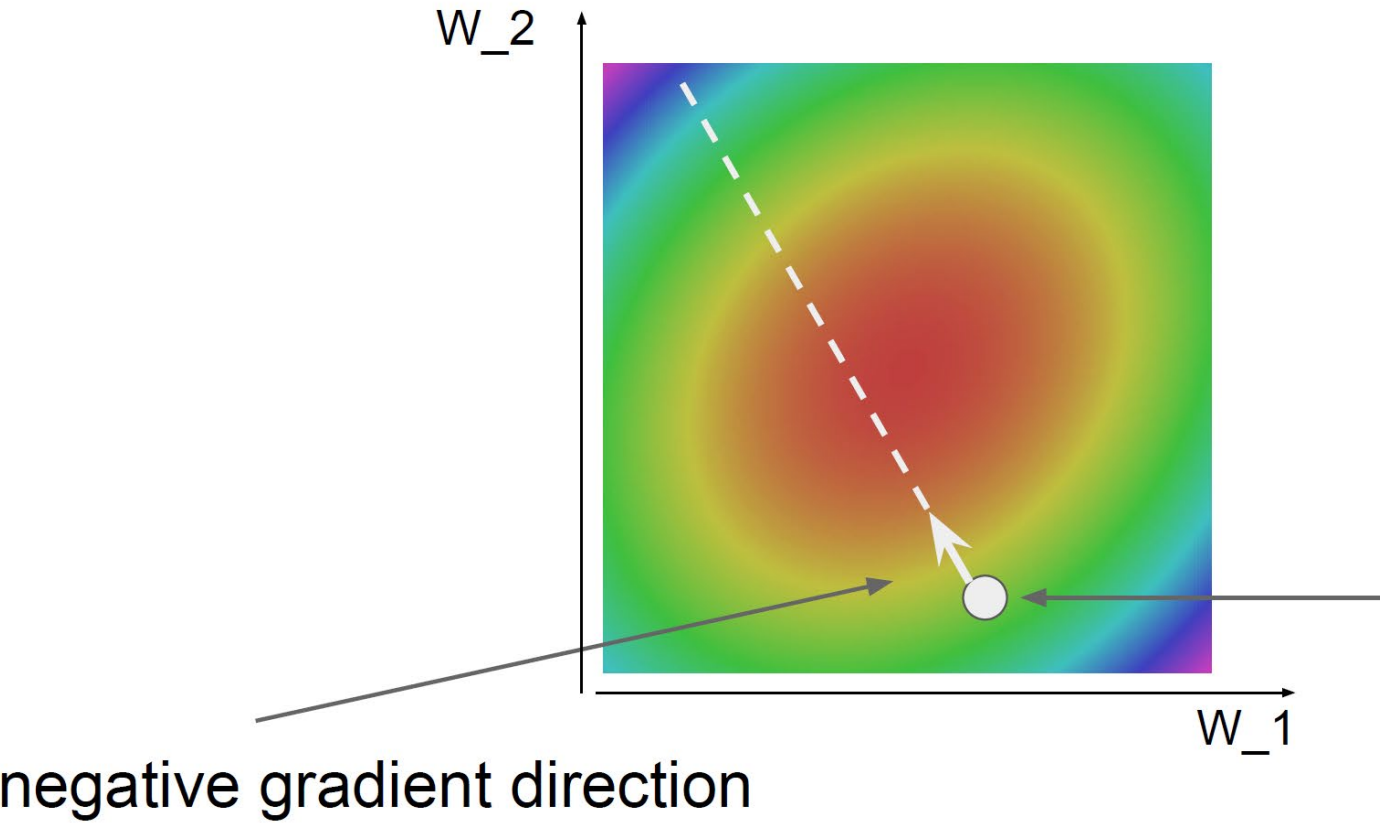
Training a Linear Classifier

Gradient Descent

```
# Vanilla Gradient Descent

while True:
    weights_grad = evaluate_gradient(loss_fun, data, weights)
    weights += - step_size * weights_grad # perform parameter update
```

Training a Linear Classifier



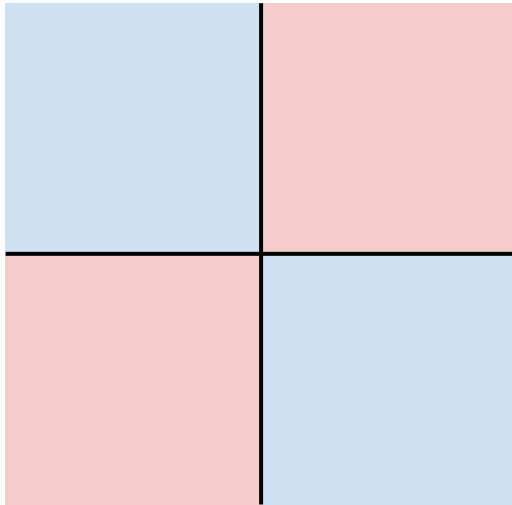
Hard cases for a linear classifier

Class 1:

number of pixels > 0 odd

Class 2:

number of pixels > 0 even

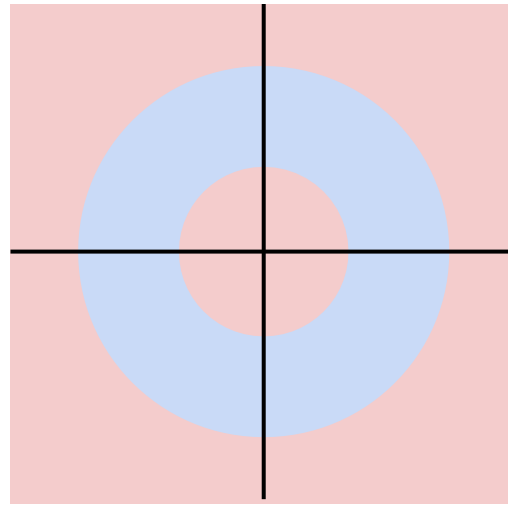


Class 1:

$1 \leq \text{L2 norm} \leq 2$

Class 2:

Everything else

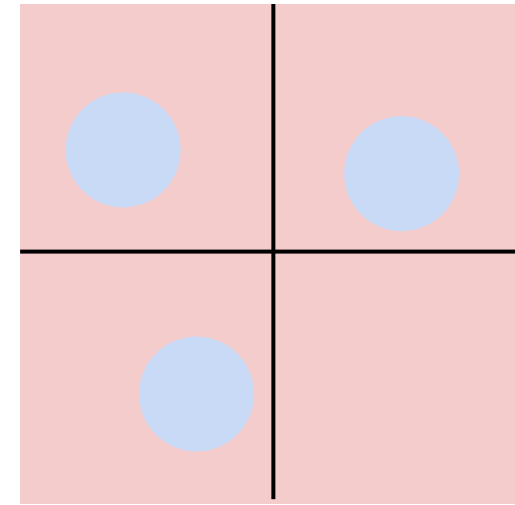


Class 1:

Three modes

Class 2:

Everything else



Linear classifier (LC) can only find hyperplanes that separate classes

A LC alone does not work for non-linear classification

How can we improve it?

From linear to non-linear classification

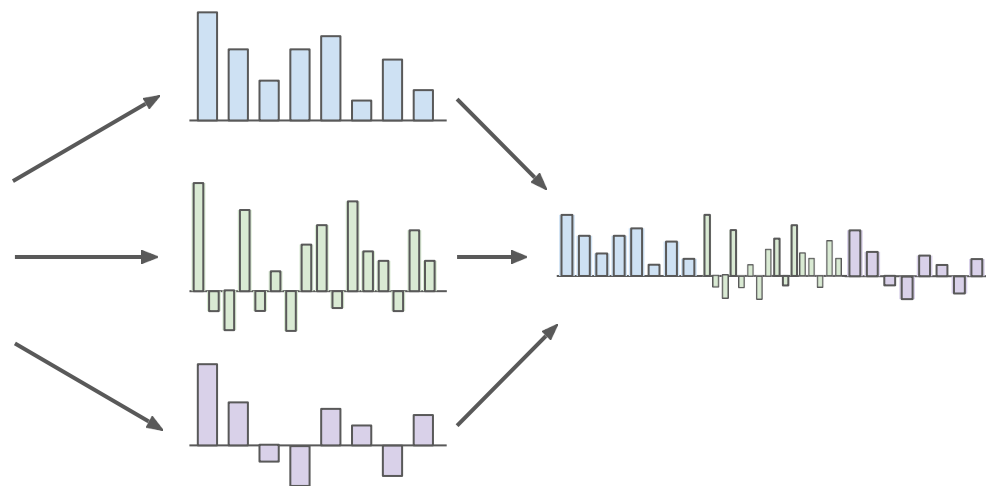
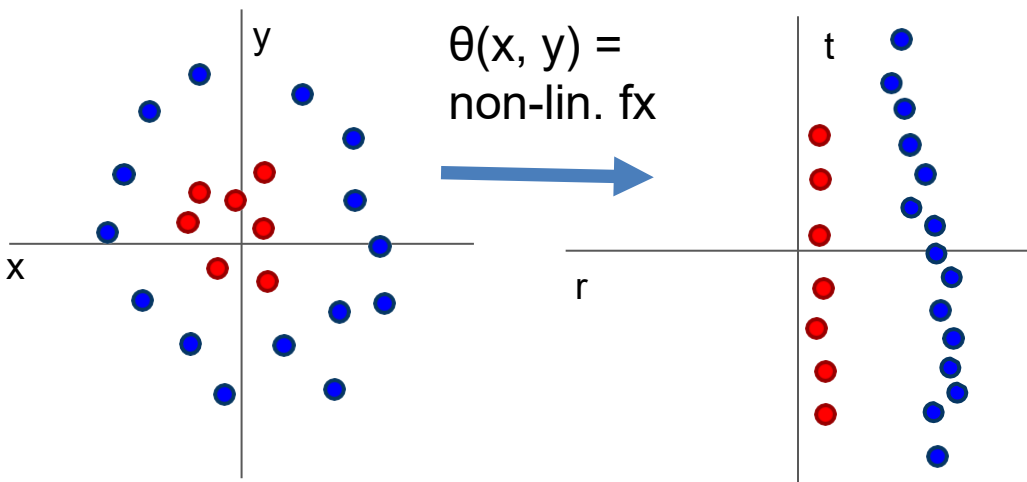
In general, we need to find a **non-linear** transformation that **maps** the raw representation (the image, the frog) into a **USER-DEFINED feature map space (usually with higher dimensionality)** which is linearly separable by hyperplanes.

The “frog” would be represented by a vector in the USER-DEFINED feature map space.

PROBLEM: it might very difficult to find the “shape”/“equation” of the non-linear transformation (because of high dimensionality features)

Cannot separate red and blue points with linear classifier

After applying feature transform, points can be separated by linear classifier in a HIGHER-dimensional feature space!

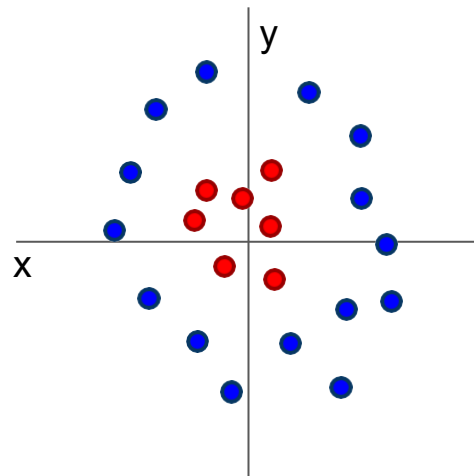


The “kernel trick”

The “Kernel trick” simplifies the problem! → we do not need to find the shape of the non-linear transformation (the coordinates of the new high-dimensional space) but it’s enough to find its “kernel” function in the feature space.

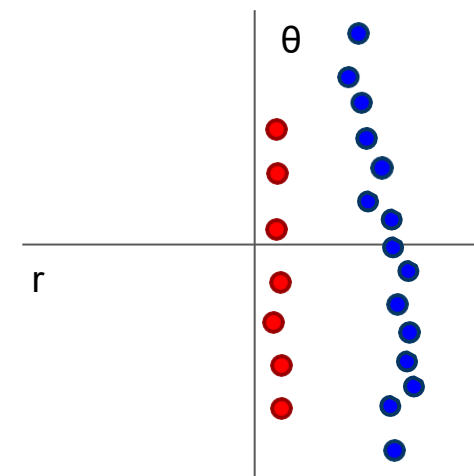
It is based on this assumption that:

- The LC algorithm does not need to know how each point of the raw input space is mapped to the “feature space”
- The LC must only know how each point compares to any other in the new feature space (i.e. after we apply the non-linear transformation). → this relationship is given by the “kernel” function (easier to define)



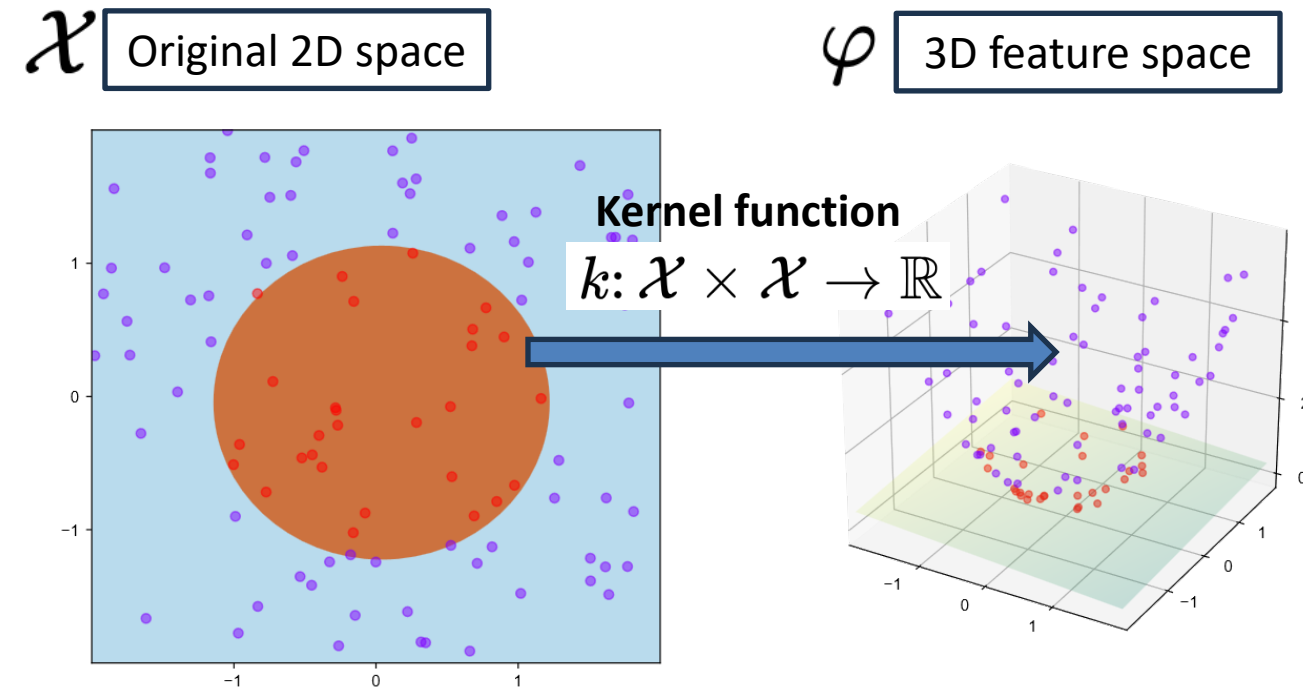
Cannot separate red
and blue points with
linear classifier

$$k(x, y) = (r(x, y), \theta(x, y))$$



After applying feature
transform, points can
be separated by linear
classifier

The “kernel trick” example



Non-linear feature map function

$$\varphi: \mathcal{X} \rightarrow \mathcal{V} \quad \varphi((a, b)) = (a, b, a^2 + b^2)$$

Associated kernel function

$$k(\mathbf{x}, \mathbf{x}') = \langle \varphi(\mathbf{x}), \varphi(\mathbf{x}') \rangle_{\mathcal{V}}$$

$$k(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{y} + \|\mathbf{x}\|^2 \|\mathbf{y}\|^2$$

An explicit representation of φ phi is not necessary as long as the space $\mathcal{V}$ is an inner product space

Kernel functions can operate in a high dimensional **implicit** feature space without even computing the coordinates of the data in that space, but rather by simply computing the inner products between the “images” $\varphi(\mathbf{x})$ of all pairs of data in the new feature space → Also computationally cheaper!

By Shiyu Ji - Own work, CC BY-SA 4.0,

<https://commons.wikimedia.org/w/index.php?curid=60458994>

Kernel trick explained visually: <https://www.youtube.com/watch?v=Q7vT0--5VII>

How to choose the right kernel?

- **Linear Kernel:**
 - Use when data is linearly separable.
 - Efficient for low-dimensional data with many features.
- **Polynomial Kernel:**
 - Suitable for data with polynomial features or feature interactions.
 - Maps data to higher-dimensional space using polynomial functions.
- **RBF (Radial Basis Function) Kernel:**
 - Ideal for complex, non-linear data.
 - Requires careful tuning of the gamma **parameter** to avoid overfitting.
- **Sigmoid Kernel:**
 - Works well with data that has a sigmoidal shape or strong nonlinearities.
 - Maps data using sigmoidal functions.
- **Other Kernels (Laplacian, ANOVA, Bessel, etc.):**
 - Less common, but useful for specific data types.

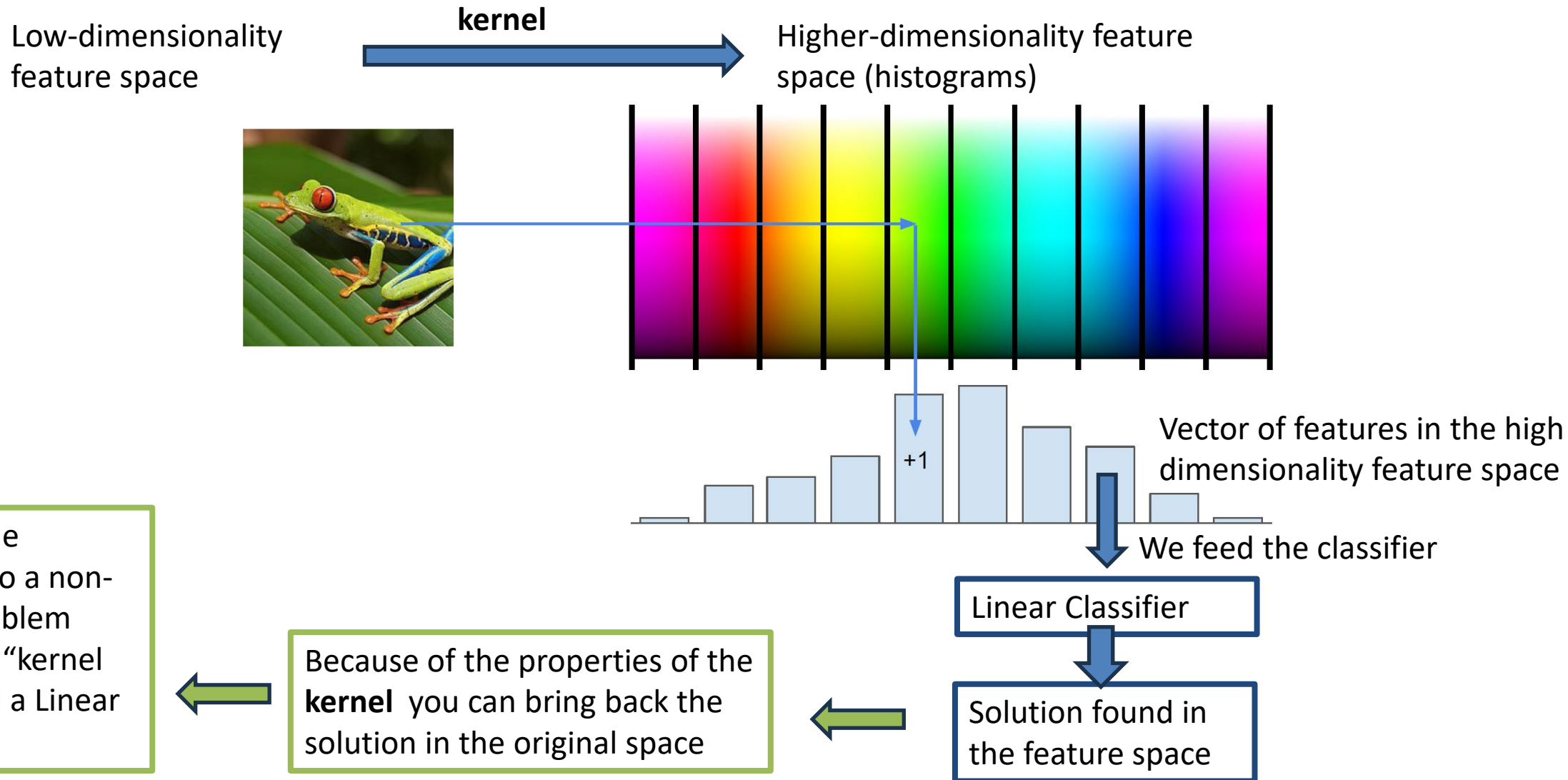
Best Practices:

- **Test multiple kernels with cross-validation.**
- **Optimize** hyperparameters using grid search, random search, or Bayesian optimization.
- Consider **computational cost** for large datasets.

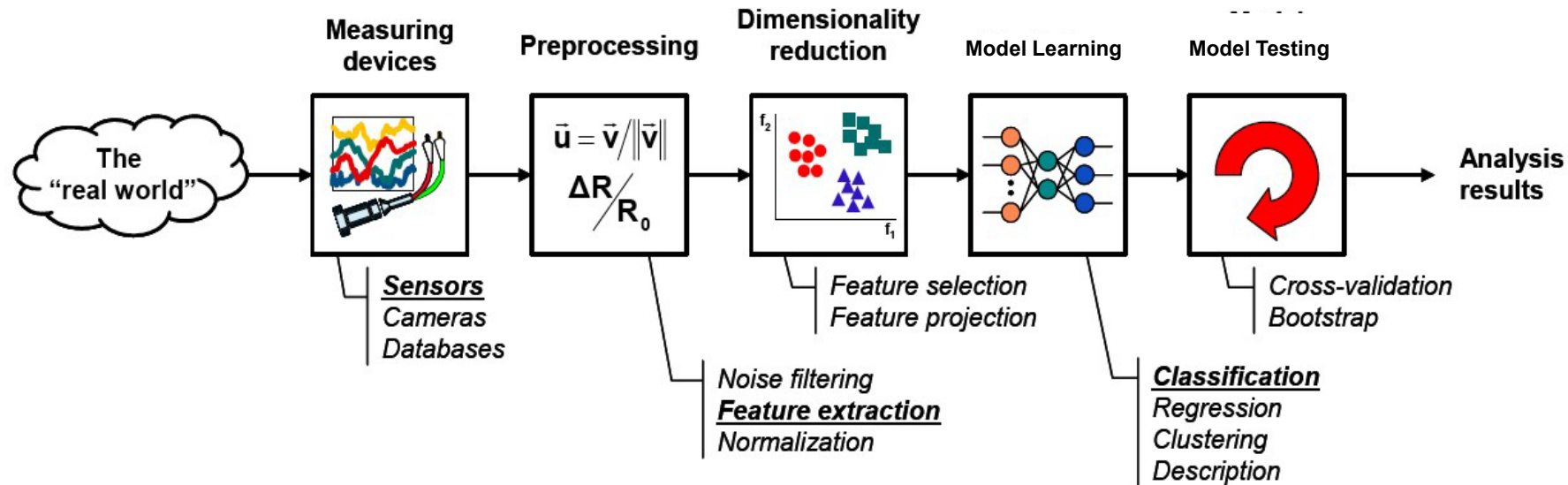
Impact of Kernel function on LC Performance

- **Separability:**
 - Determines how well data can be separated by a hyperplane.
 - Linear kernel for separable data; nonlinear kernels (RBF, polynomial, sigmoid) for non-separable data.
- **Complexity:**
 - Nonlinear kernels create more complex decision boundaries.
 - Higher complexity can improve accuracy but risks overfitting if not regularized.
- **Computational Cost:**
 - Linear kernel: Efficient for large datasets.
 - Nonlinear kernels: More expensive, especially in high-dimensional spaces (e.g., RBF).
- **Generalization:**
 - A well-chosen kernel ensures good performance on unseen data.
 - Risk of poor generalization if the kernel is too specific to the training data.
- **Hyperparameter Tuning:**
 - Kernels have specific hyperparameters (e.g., gamma for RBF).
 - Proper tuning is essential to balance underfitting vs. overfitting.

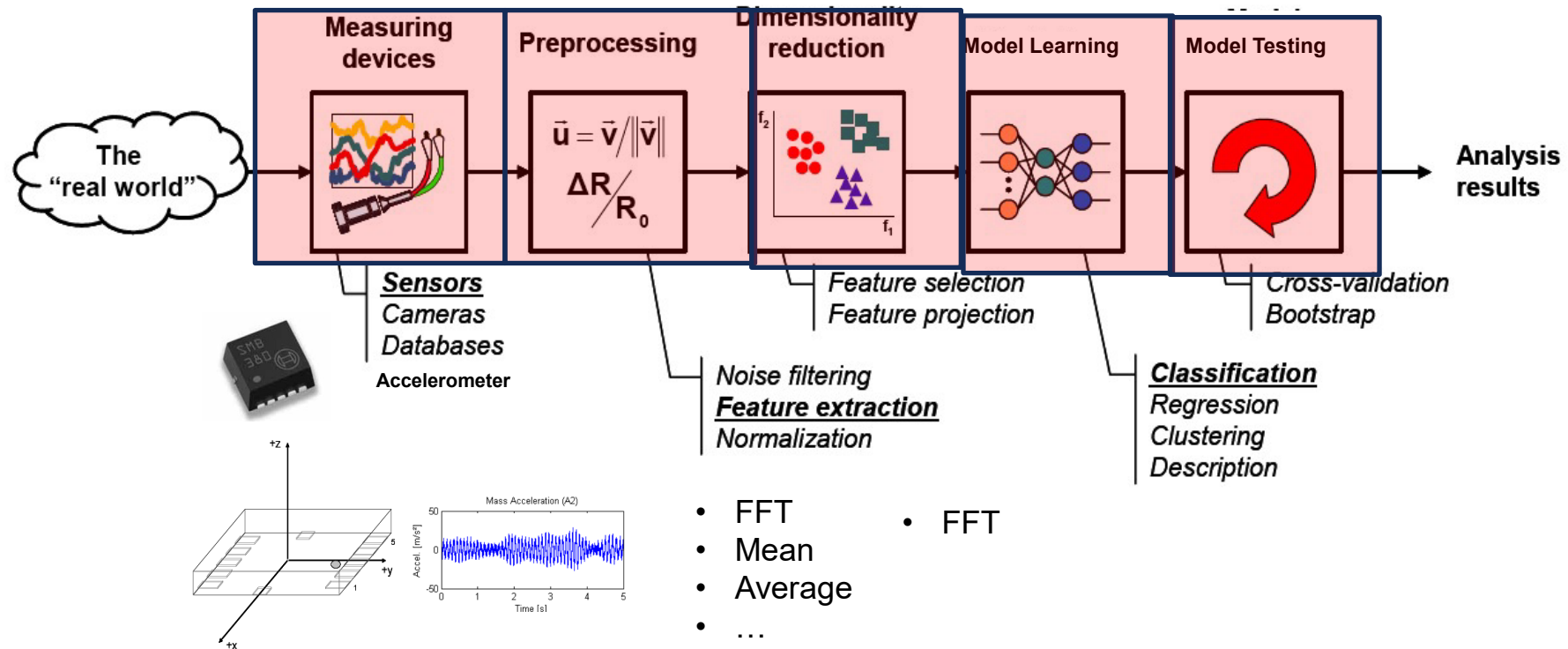
Example: the Color Histogram



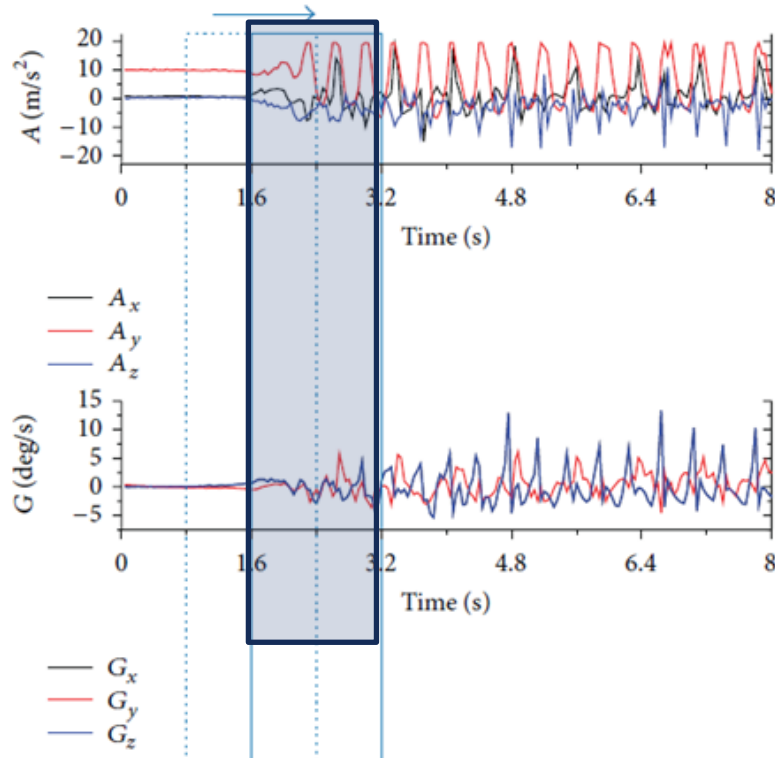
The Learning Process and the ML Datapath



Example: Activity Recognition



From Raw data to features extractions

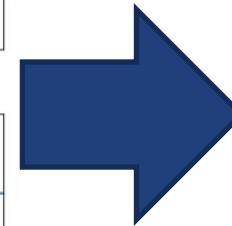
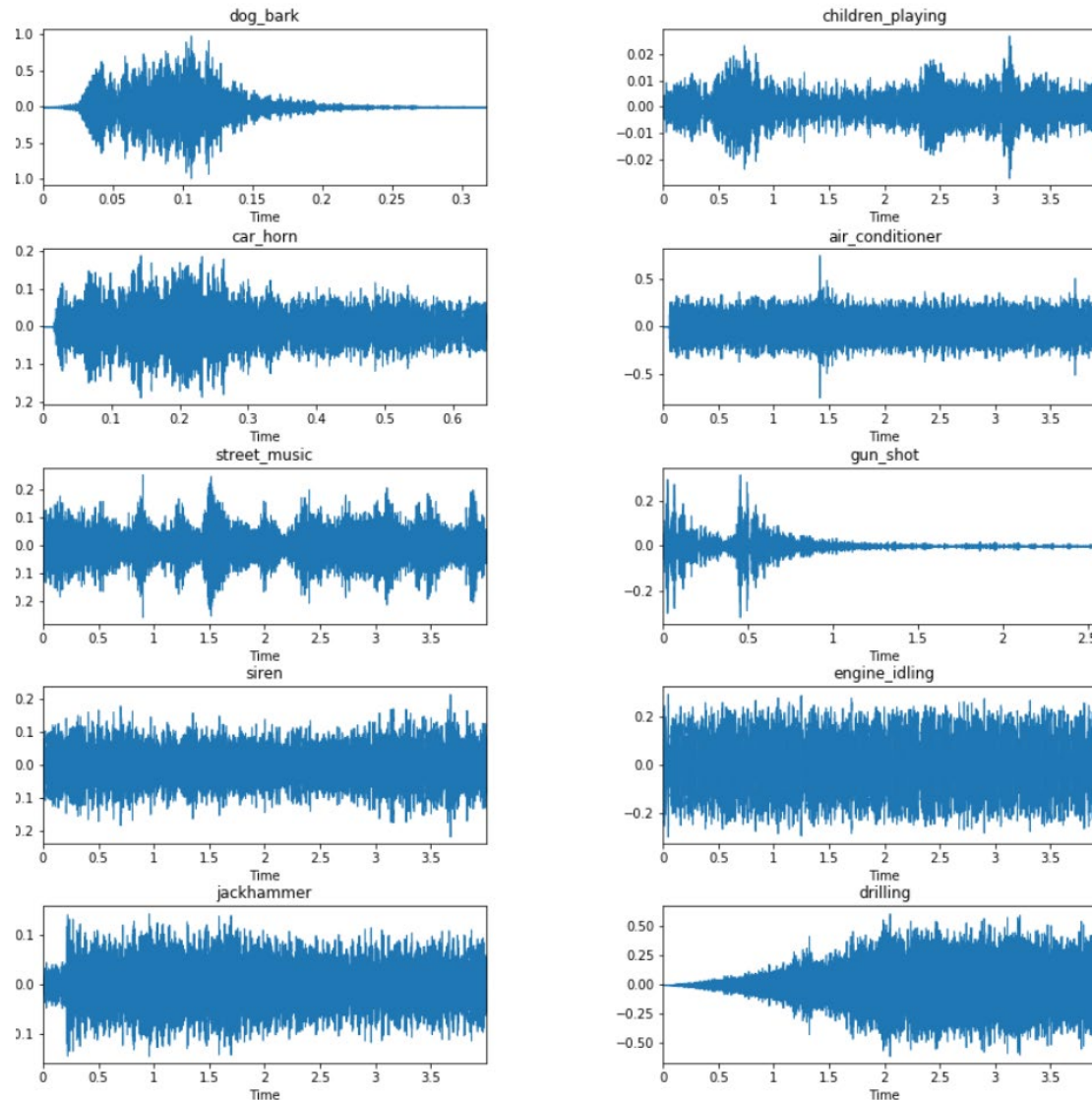


- Sample rate (10-100Hz)
- Windows (0.5-2s)
- Data to be stored
- Action to be detected
 - ✓ Activities
 - ✓ Falls
 - ✓ Calories
 - ✓ Gestures

Typical features for motion (Accelerometer)

1. Area under Curve
2. Variance
3. Mean
4. Median (less sensitive to outliers)
5. Entropy
6. Correlation
7. FFT peaks
8. Energy
9. Standard deviation
10. Fluctuation
11. Mean gradient
12. Mean crossing rate
13. Absolute gradient
14. Spectral fluctuation
15. Center of gravity
16. Bandwidth
17. Band energy ratio
18. spectral roll off frequency
19. Root mean square
20. Number of zero crossing
21. IQR
22. Low frequencies power
23. High frequencies power
24. Frequency entropy
25. Spectral Entropy
26. Cepstral coefficients ($\text{FFT}(\log(\text{FFT}(x)))$)
27. Log FFT frequency bands
28. Linear FFT frequency bands
29. Integration
30. Mean crossing

Audio Signals for different Classes



Some features?

for audio

- zero-crossing rate

$$\text{ZCR} = \sum_{n=1}^{N-1} |\text{sgn}(x_n) - \text{sgn}(x_{n+1})|$$

- fluctuation of signal amplitude $\text{FLUC} = \frac{\mu_x}{\sigma_x}$

- frequency domain

- ▶ F(n) absolute value of the FFT coefficients
- ▶ spectral Center of Gravity CG
- ▶ bandwidth BW
- ▶ spectrum energy density
- ▶ autocorrelation r

$$\text{BW} = \sqrt{\frac{\sum_{n=1}^N (n - \text{CG})^2 \cdot F(n)^2}{\sum_{n=1}^N F(n)^2}}$$

$$S_{xx}^E(e^{j\omega}) = |X(e^{j\omega})|^2$$

$$\text{CG} = \frac{\sum_{n=1}^N n \cdot F(n)}{\sum_{n=1}^N F(n)}$$

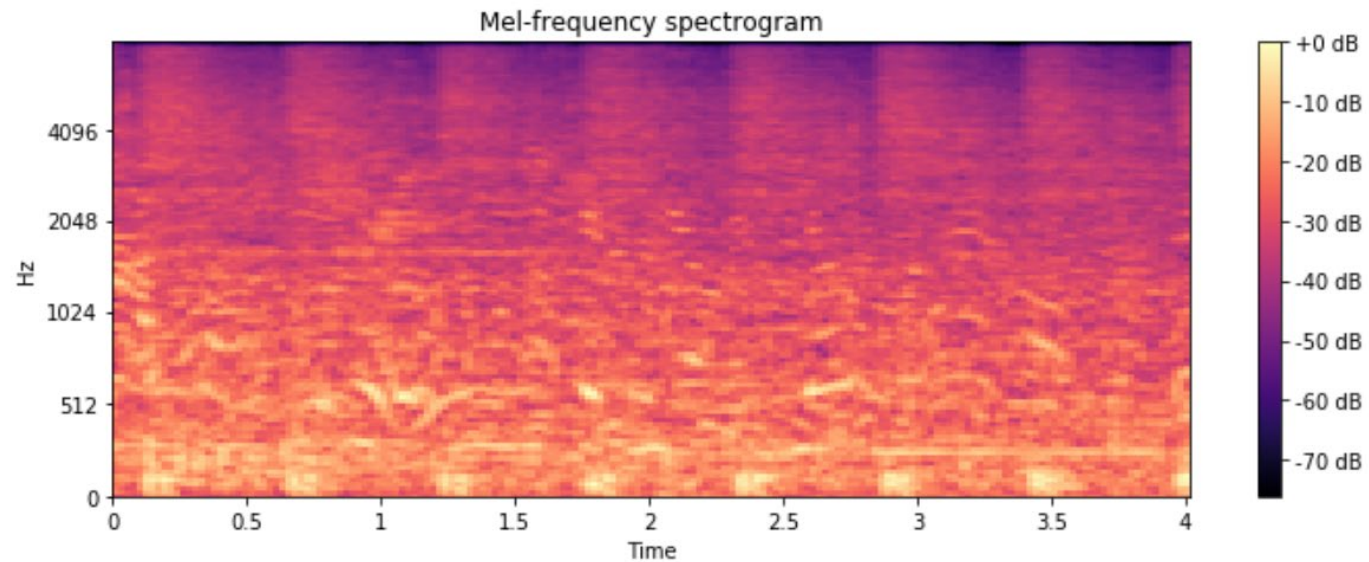
$$r_{xx}^E(m) = \sum_{n=-\infty}^{+\infty} x^*(n) \cdot x(n+m)$$

openSMILE:

Speech & Music Interpretation by Large-space Extraction

- ▶ Music Information Retrieval and Speech Processing; can extract more than 5000 features
 - ▶ from <http://www.audeering.com/research/opensmile>
 - ▶ http://web.stanford.edu/class/cs224s/hw/openSMILE_manual.pdf

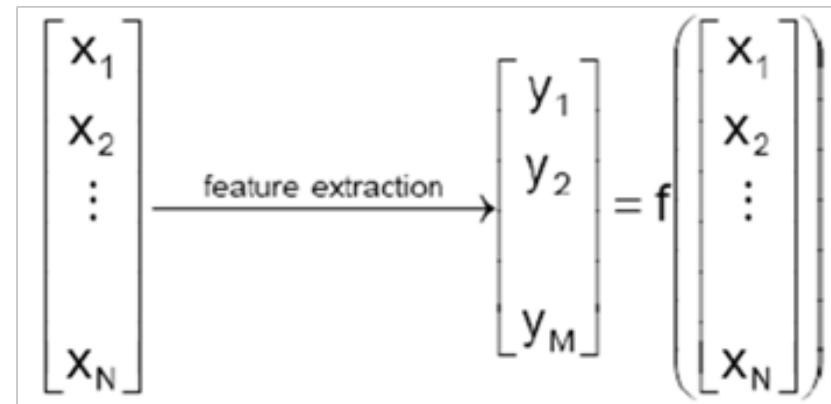
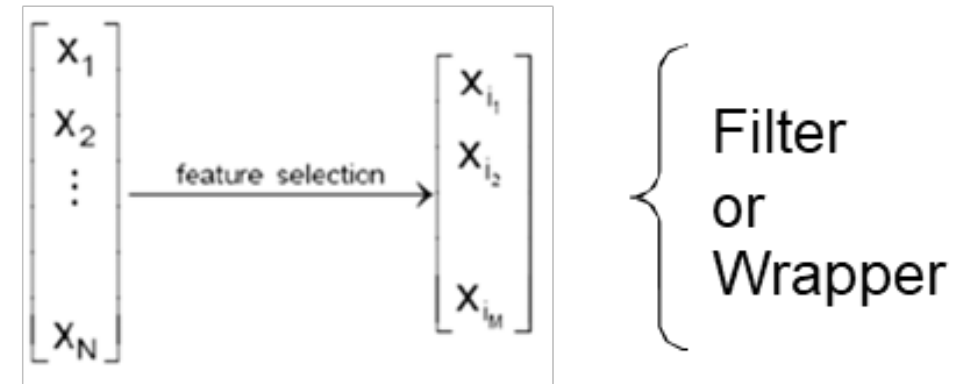
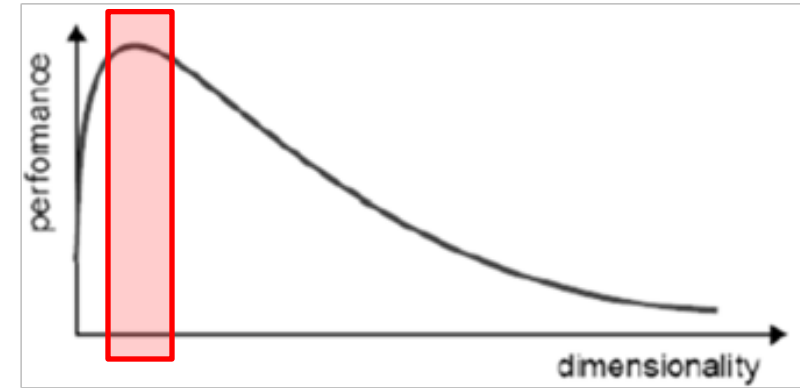
One of most popular and used for Audio



A **spectrogram** is a visual representation of the spectrum of frequencies of a signal as it varies with time. A nice way to think about spectrograms is as a stacked view of periodograms across some time-interval digital signal.

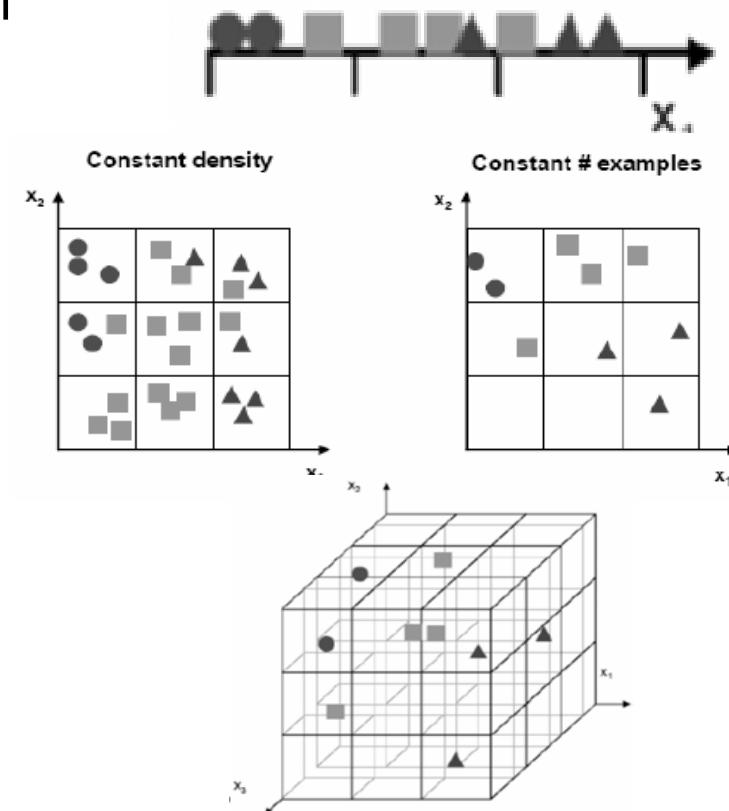
Exploring the dimensionality

- For a given sample size, there is a maximum number of features above (below) which the performance of our classifier will degrade rather than improve” [2].
also more than one optima are possible
- Feature selection
 - ▶ subset of all features
- Feature extraction
 - ▶ creating a subset of new features by combinations of the existing features



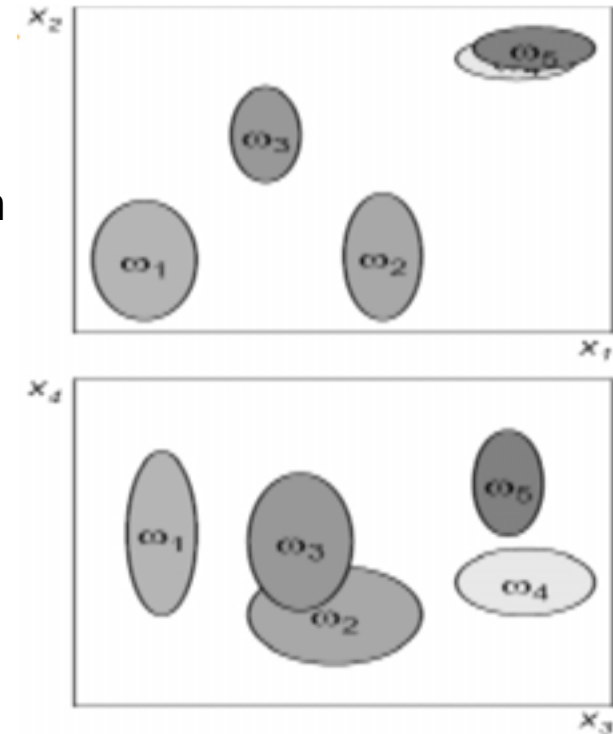
Feature Selection: Curse of dimensionality

- Example: 3-class pattern recognition problem
- 1 feature
 - 9 data points, start with feature space divided into 3 equally spaced bins
- Better: 2 features
 - maintain bin density/feature: $3^2=9$ bins
 - maintain #data results in a sparse scatter plot
- 3 features even worse
 - bin number: $3^3=27$

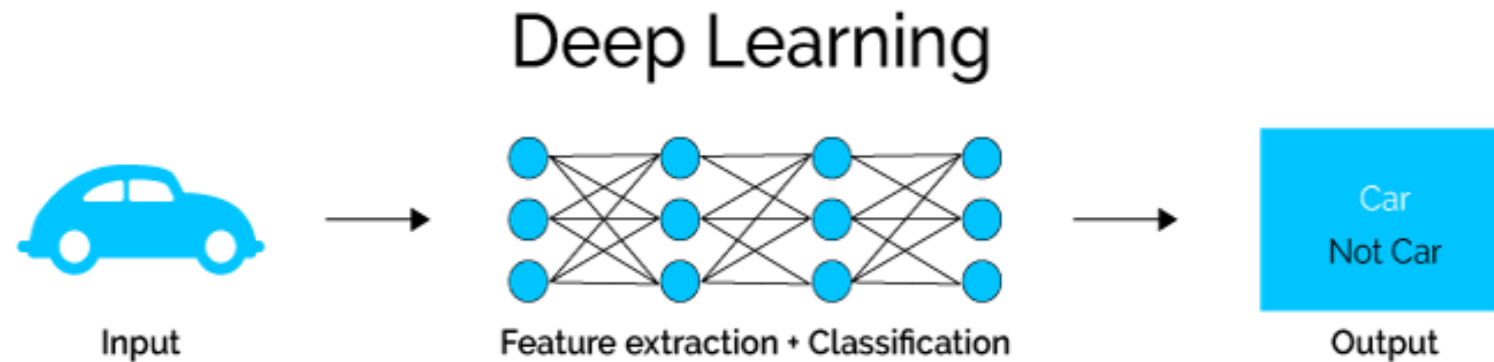
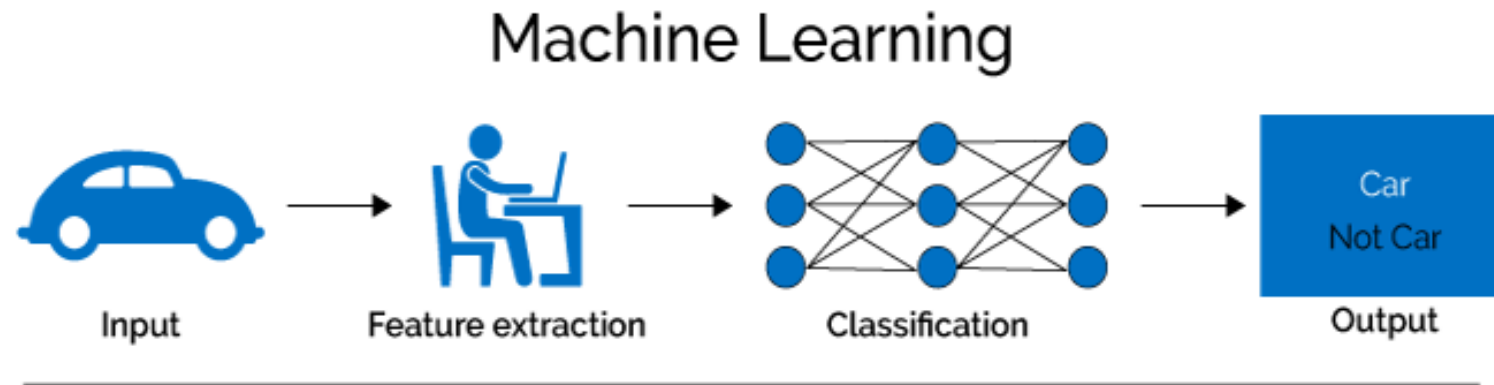


Naïve sequential feature selection: the wrong way

1. evaluate each individual feature separately
 2. select M features with the highest scores
- works very rarely: does account for feature dependence ?**
- example: 4-dimensional pattern recognition problem with classes, select the 2 best features
 - ranking using any reasonable objective function:
 - $J(x_1) > J(x_2) \cong J(x_3) > J(x_4)$
 - x_1 can discriminate $\omega_1, \omega_2, \omega_3$; similar to x_2, x_3
 - x_4 is the worst feature, because it can only discriminate ω_4 and ω_5**
 - but the optimal feature subset is $\{x_1, x_4\}$
 - x_1 can discriminate $\omega_1, \omega_2, \omega_3$
 - x_4 can discriminate ω_4, ω_5
 - Consequence: **not looking for the m best features separately, but for the best m features as group**

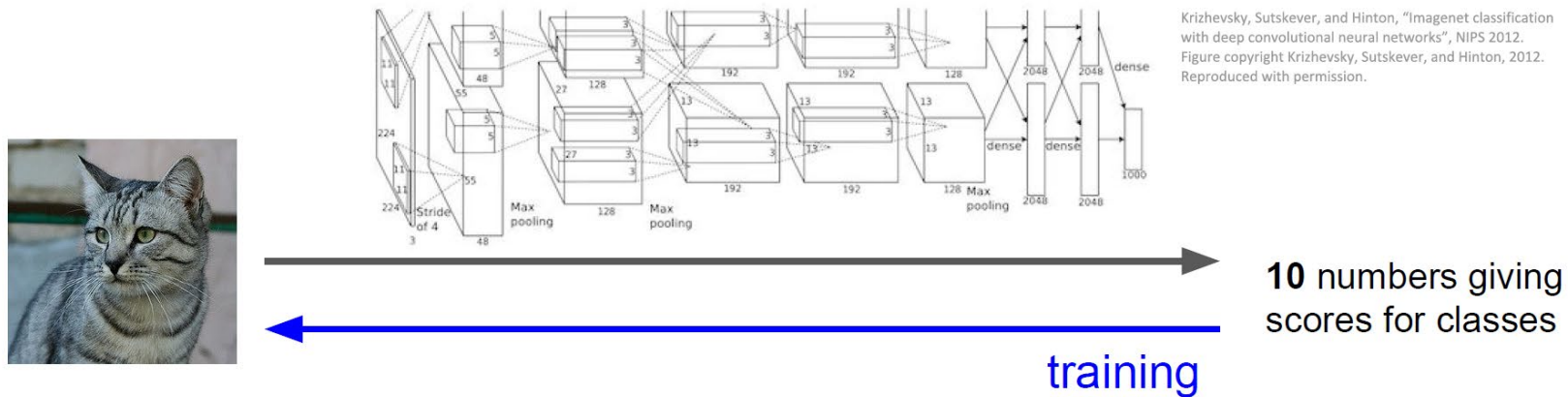
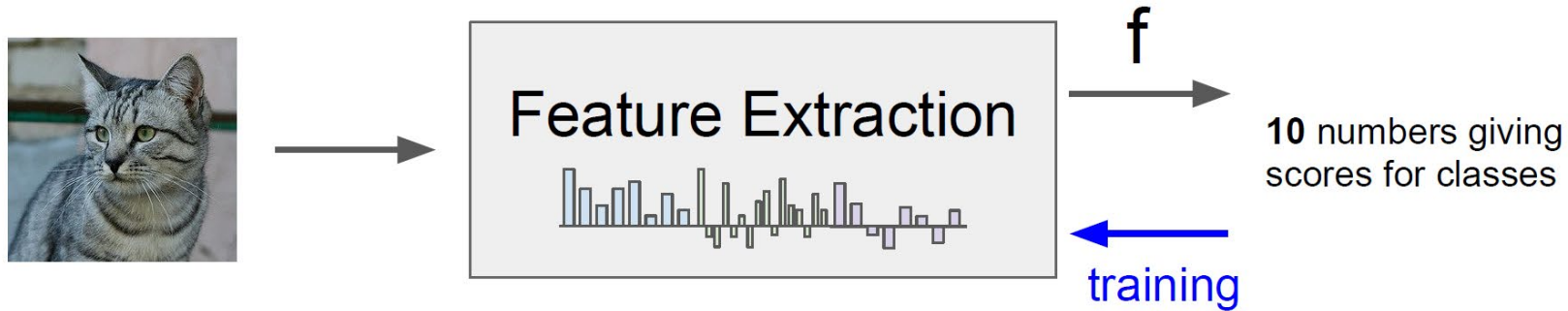


Feature selection is a hard job...

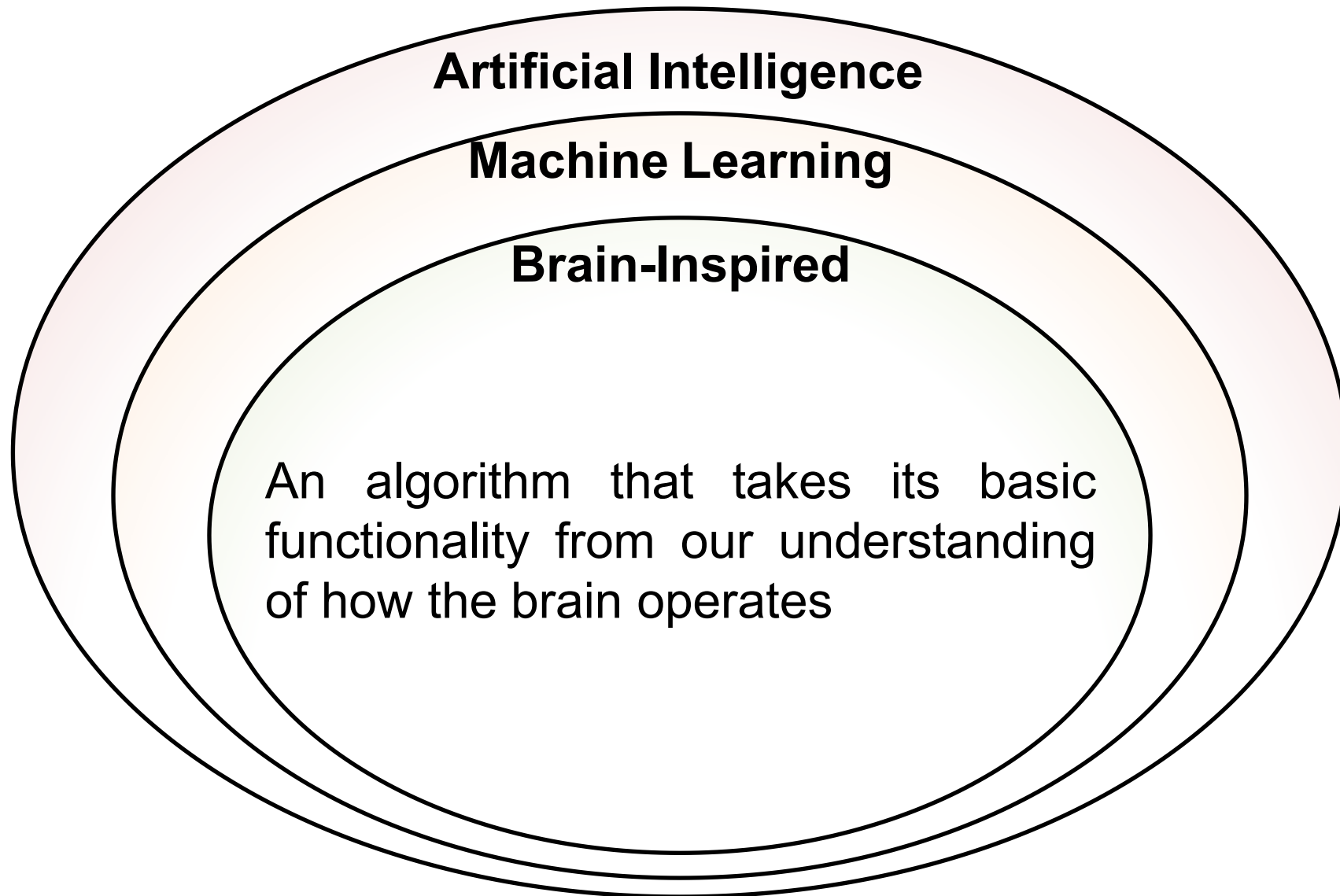


Deep Networks

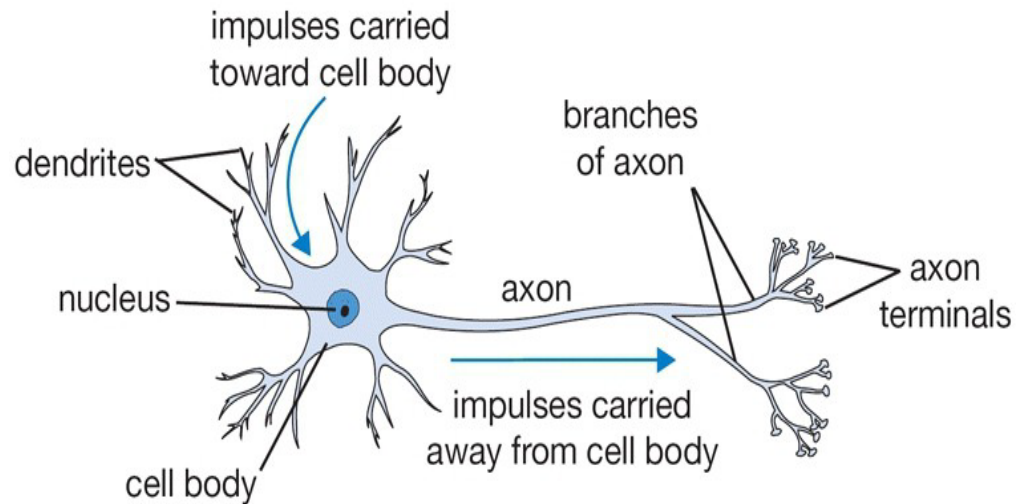
Trained feature extractors



Brain-Inspired Machine Learning

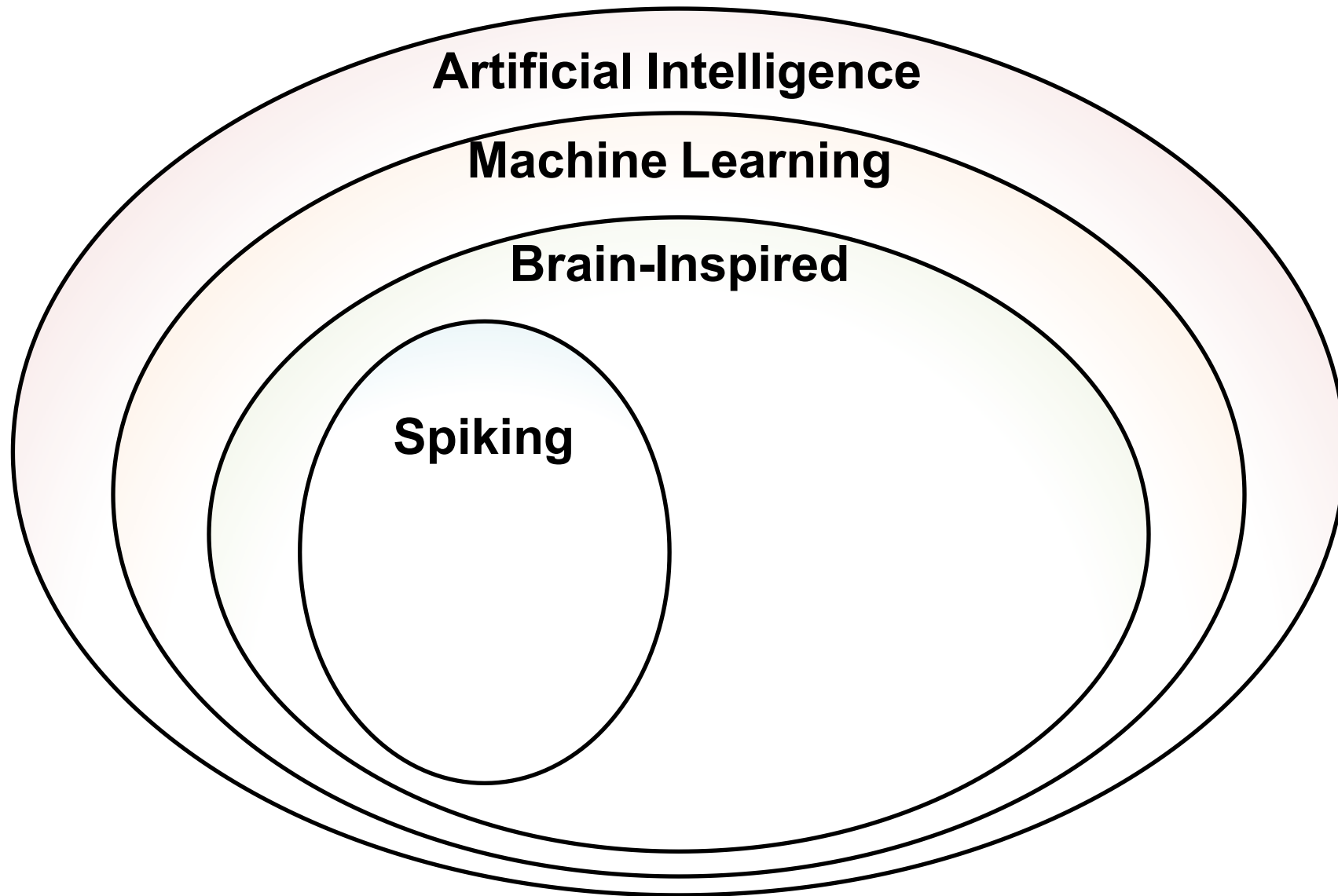


How Does the Brain Work?

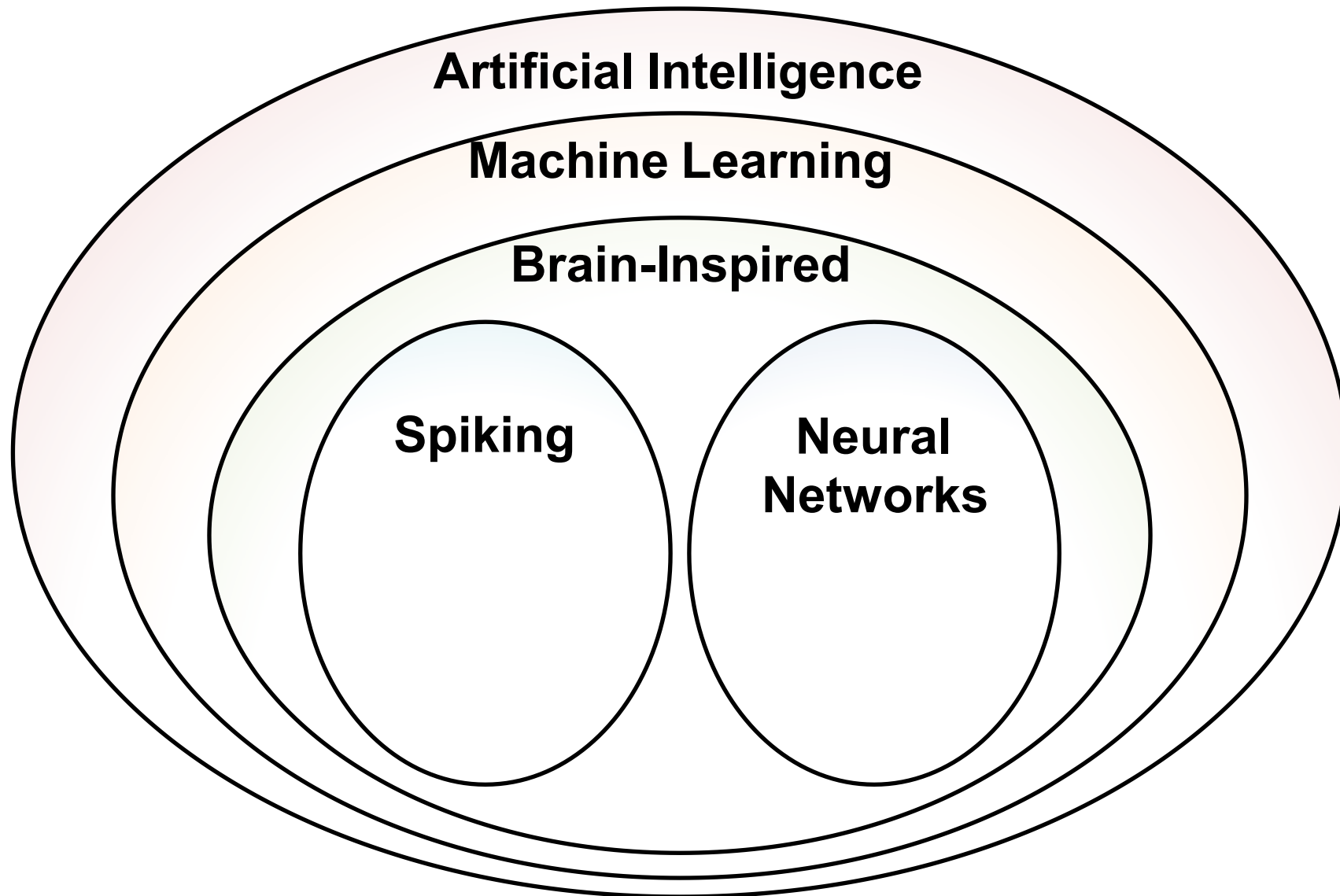


- The basic computational unit of the brain is a **neuron**
→ 86B neurons in the brain
- Neurons are connected with nearly $10^{14} - 10^{15}$ **synapses**
- Neurons receive input signal from **dendrites** and produce output signal along **axon**, which interact with the dendrites of other neurons via **synaptic weights**
- Synaptic weights – learnable & control influence strength

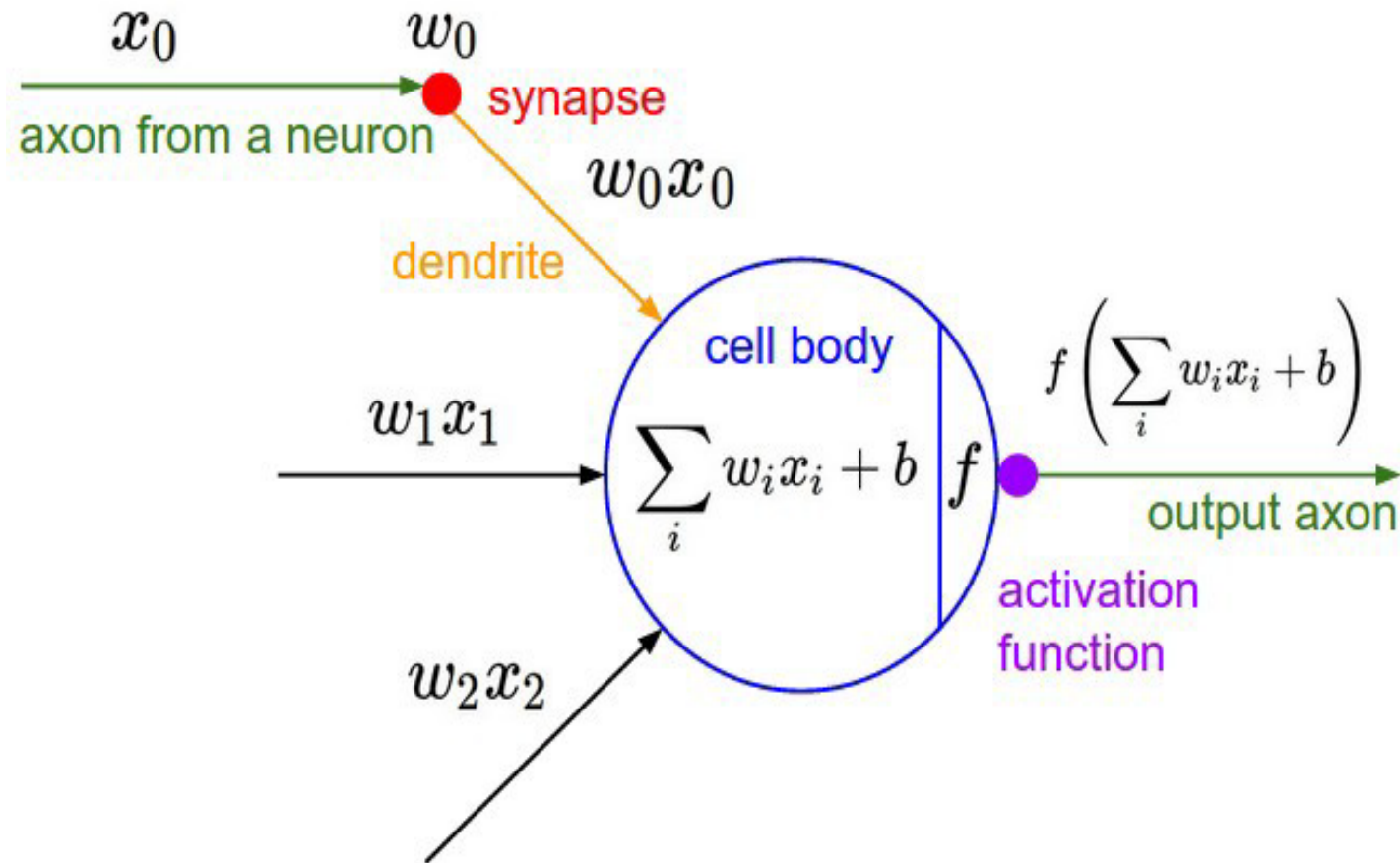
Spiking-based Machine Learning



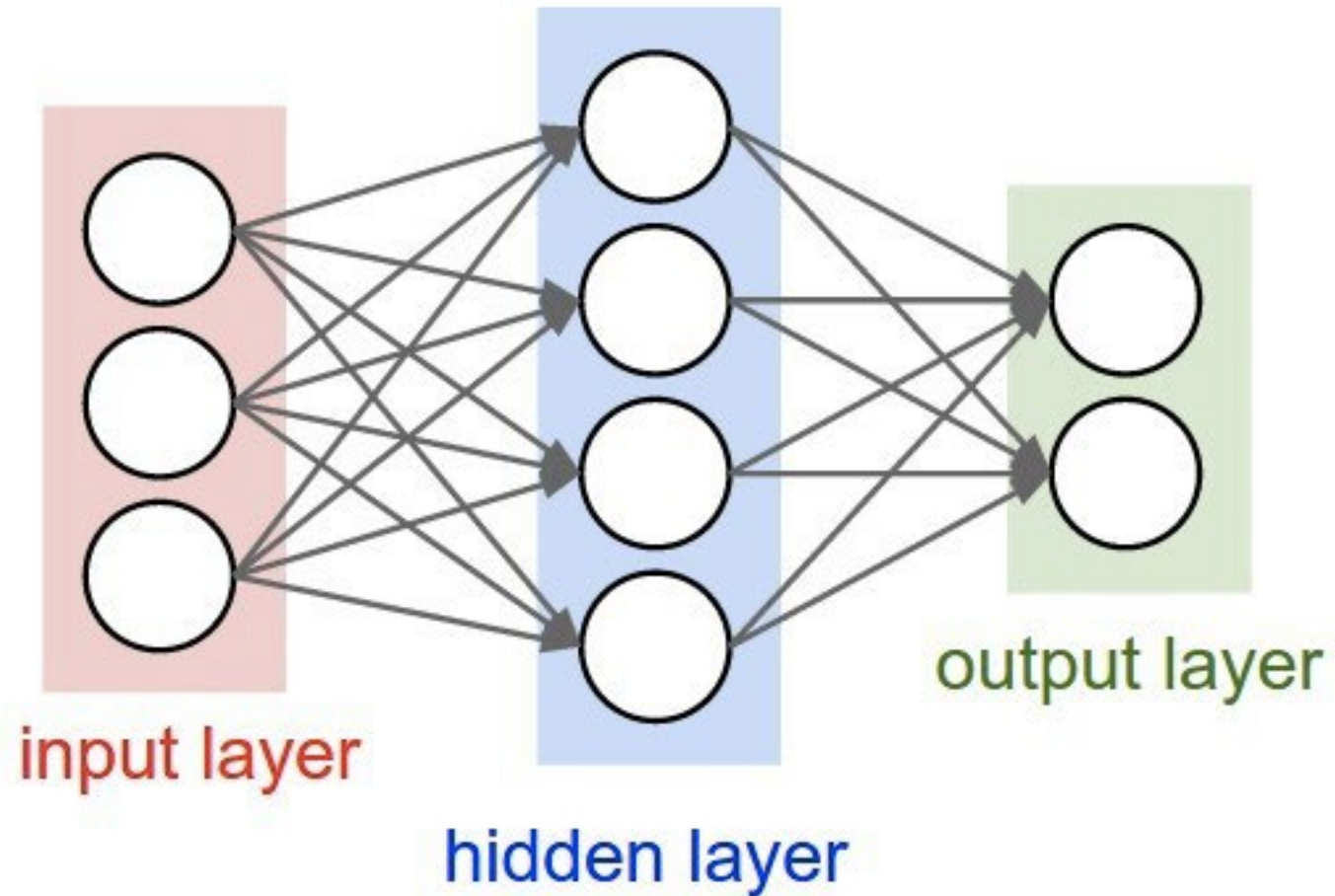
Machine Learning with Neural Networks



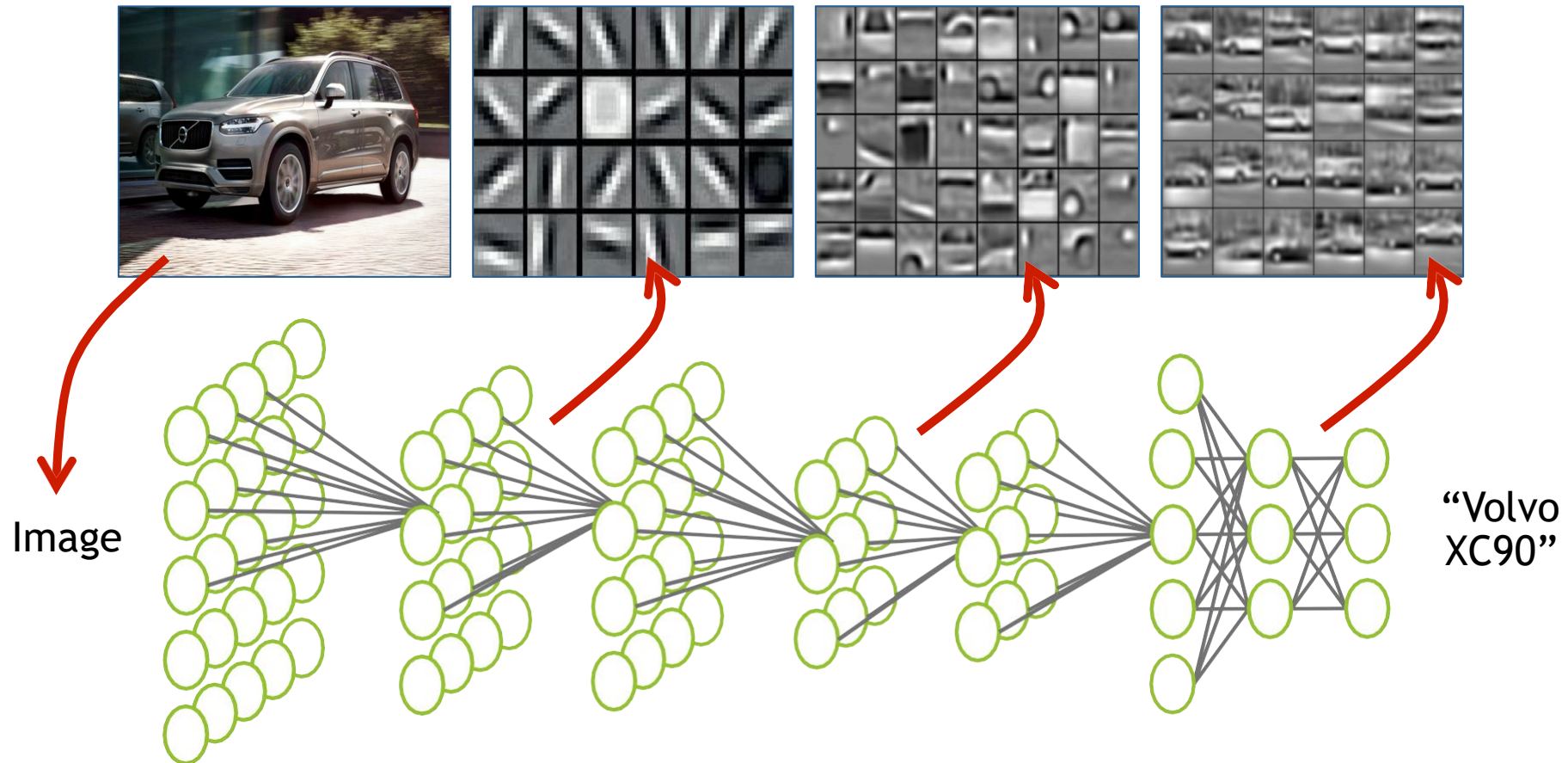
Neural Networks: Weighted Sum



Many Weighted Sums



What is a Deep Network?



Deep Learning

